Lab 05 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

Question 1 — 950 MB refused by security

Answer. (1) **The source code is in the image**: anyone who pulls the image reads the application — and often local configuration files. (2) **The attack surface**: JDK, Maven, `git`, `curl`, a full shell, hundreds of Debian packages — as many tools for an attacker who obtains code execution, and a 300-CVE scan report nobody will handle. (3) **The `~/.m2` repository** contains the downloaded artefacts and, frequently, a `settings.xml` with the credentials of the private Maven repository. The hardest to fix without multi-stage is (2): you can `rm` the sources and `.m2` (badly: the layers keep them, lab 02), but you cannot remove the JDK from an image that starts from a JDK image.

Why. The final image is the last `FROM` plus what you add to it; you cannot "subtract" the base image. Only a second `FROM` on a minimal base, with `COPY --from`, changes the base.

Nuance. Size itself has an operational cost (pull time on an incident, registry storage), but it is the weakest argument in front of a security officer — content matters more than weight.

Example.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version; ls ~/.m2 2>/dev/null'
```

Question 2 — Three `FROM`s

Answer. The **last** `FROM` produces the final image; the two others are temporary environments, destroyed at the end of the build (only their cache remains). If no `COPY --from` (nor a later `FROM`) references the second stage, Buildah **does not build it at all**: it ignores it. You see it in the output: the stages are numbered `[1/3]`, `[2/3]`, `[3/3]`, and the number of the unused stage never appears.

Why. The engine first computes the dependency graph between stages from the `--from`s, then only builds what leads to the target (the last stage, or `--target`).

Nuance. BuildKit does the same and additionally builds independent stages in parallel. An "unused" stage has a legitimate use: a test stage (`RUN mvn test`) that is only built with `--target test` in CI.

Example.

```
podman build -f Dockerfile.unused -t u . 2>&1 | grep STEP     # [2/3] and [3/3] only, never [1/3]
```

Question 3 — 420 MB and the sources

Answer. `COPY --from=build /app /app` copies **the whole working directory** of the `build` stage: `Api.java`/`src`, `pom.xml`, `target/` with its classes, and the JAR. Fix: copy only the artefact.

```
COPY --from=build /app/target/api.jar /app/api.jar
```

(and adapt the `ENTRYPOINT` to `/app/api.jar`.)

Why. Multi-stage filters nothing by itself: it puts into the final image only what `COPY --from` asks for. Asking for a folder is asking for its whole content.

Nuance. Even once fixed, does the image still contain a `.m2`? No — `.m2` is in `/root` of the build stage, not in `/app`. But that is luck, not a guarantee: the rule is to copy **named files**.

Example.

```
podman run --rm --entrypoint ls api-multi:1.0 /src     # No such file or directory: good sign
```

Question 4 — `ng serve` in production

Answer. Four reasons: (1) `ng serve` is a **development** server — unoptimised, without compression or HTTP caching, explicitly documented as not intended for production; (2) the image contains **Node, the sources and `node_modules`** (often > 1 GB): attack surface and code leak; (3) the build is not done **once** but at every start, in "watch" mode, with *source maps* enabled; (4) no separation between the application and its tooling — a vulnerable development dependency is in production. Instead: `ng build --configuration production` in a Node stage, then `COPY --from` of the `dist/…/browser` folder into an `nginx:alpine` image with an nginx configuration that returns `index.html` for Angular routes.

Why. A compiled Angular front end is static. Serving it only needs a file server; everything else is build, which belongs to the discarded stage.

Nuance. There is one case where Node stays in production: server-side rendering (Angular SSR / Universal). That is then **another** application, with its own Dockerfile, and still not `ng serve`.

Example.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'   # 64 MB against 167 MB (without node_modules!)
```

Question 5 — `UnsatisfiedLinkError` after Alpine

Answer. PDF generation most probably relies on a **native library** (`.so`) compiled for `glibc` — fonts, rendering, compression. Alpine provides `musl`: the loader refuses the library, Java throws `UnsatisfiedLinkError`. The command that would have shown it: `ldd --version` in each image (`musl libc` versus `GLIBC`). The migration should have been tested with the **real jobs** (not only `/actuator/health`), on a staging environment, with a rollback plan — and the decision taken component by component.

Why. A native library is bound to a specific `libc`; it is not portable bytecode. Two weeks of delay, because PDF generation perhaps only runs at month end.

Nuance. Alternatives exist: the `eclipse-temurin:21-jre-ubi9-minimal` image (Red Hat, `glibc`, ~100 MB), or installing `gcompat` on Alpine (fragile). And this is not specific to Java: Python, Node with native modules, have exactly the same trap.

Example.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --version 2>&1 | head -1'   # musl libc
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version | head -1'              # GLIBC 2.xx
```

Question 6 — Multi-stage and secrets

Answer. An `rm` creates a layer that hides the file; the layer that wrote it stays in the image (lab 02). A discarded stage, on the other hand, is **not** in the final image: none of its layers is there. If the secret was only written in a discarded stage, it exists nowhere in the published artefact. Multi-stage does not protect if the secret is **copied** into the final stage (`COPY --from=build /app` with the secret inside), or if the artefact itself absorbed it (an `application.yml` with a password packed into the JAR), or if the secret passes through an `ARG` of the final stage (visible in `history`).

Why. Final image = layers of the last `FROM` + layers produced by its instructions. A previous stage only contributes what a `COPY --from` extracts from it.

Nuance. The modern solution is `RUN --mount=type=secret`: the secret is available during a single instruction, in any stage, without ever becoming a layer. Multi-stage remains the structural guarantee, the *secret mount* the per-instruction guarantee.

Example.

```
podman build --secret id=pw,src=pw.txt -f Dockerfile.secret -t sec .
podman run --rm sec ls /run/secrets      # No such file or directory
```

Question 7 — JAR as a block versus layers

Answer. (a) One 50 MB layer that changes at every build: the `push` and every `pull` transfer **50 MB**. (b) Four layers: dependencies (~45 MB, unchanged), loader (~1 MB, unchanged), snapshots (0), application (~5 MB): deployment transfers **~5 MB**. Ratio 10. (a) remains acceptable because 50 MB on a datacenter network takes one second, because the JRE (180 MB) is shared anyway, and because (b) adds a stage, a different `ENTRYPOINT` (`org.springframework.boot.loader…` or `java -jar` on the folder) and complexity to explain.

Why. Transfer is differential per layer; what counts is the size of the layer that changes, not that of the image.

Nuance. (b) becomes worthwhile when you deploy often on many nodes, or over a slow network (edge, remote sites). And the principle applies without Spring: separating `lib/` (stable) and `classes/` (volatile) is enough.

Example.

```
podman history api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6   # one 50 MB layer, or four layers
```

Question 8 — 90 seconds turned into 7 minutes

Answer. The new agent has an **empty cache**: the build cache (layers) lives on the machine that builds, and a new agent — or an ephemeral agent recreated at every pipeline — starts from zero. The 5 minutes of `dependency:go-offline` are therefore paid again. Two mechanisms: (1) a **cache**

mount (`RUN --mount=type=cache,target=/root/.m2`), which keeps the Maven repository on the agent between builds, even when `pom.xml` changes; (2) a **remote cache** — `--cache-from` / `--cache-to` towards the registry — which lets a new agent fetch the layers of a previous build. With rootless Podman, the cache (layers and *cache mounts*) is in `~/.local/share/containers/storage` of the user who builds: an agent that runs each job under a different user or `home`, or that destroys its `home`, never has a cache.

Why. "Nothing changed" is true on the source side, false on the cache side: the cache is a local state of the machine, not a property of the Dockerfile.

Nuance. Ephemeral agents are intentional (isolation, reproducibility); the answer is to make the cache **explicit and external**, not to keep long-lived agents. And a `--no-cache` in the pipeline "to be safe" produces exactly this symptom, permanently.

Example.

```
podman build --cache-to registry.internal/myapp/api-cache --cache-from
registry.internal/myapp/api-cache -t api:1.5 .
podman info --format '{{.Store.GraphRoot}}'    # where this user's cache lives
```

Question 9 — What you lose with distroless

Answer. (1) **`podman exec -it … sh`**: no shell, so no interactive exploration, no `cat` of a configuration file, no `curl localhost:8080/actuator`. Compensation: exposed observability endpoints (`/actuator/health`, `/info`, `/env`), structured and complete logs on `stdout`, and `podman cp` to extract a file. (2) **Diagnostic tools** (`jcmd`, `jstack`, `ps`, `netstat`): nothing to take a *thread dump* or see the sockets. Compensation: a debug *sidecar* container that shares the namespaces (`podman run --pid=container:api --network=container:api debug-image`), or JMX/Actuator tools (`/actuator/threaddump`) exposed on the internal network.

Why. Everything the operator uses to get into a container, an attacker uses too. Distroless removes both at once; observability must therefore move **out** of the image.

Nuance. Distroless images exist in a `:debug` variant with a busybox shell — useful in staging, forbidden in production. And Kubernetes offers `kubectl debug` with ephemeral containers for exactly this need.

Example.

```
podman exec d sh -c ls          # executable file `sh` not found
curl -s localhost:18082/actuator/health     # observability goes through HTTP
```

Question 10 — The cache mount, `VOLUME`, and `# syntax=`

Answer. The data lives in a **cache folder managed by the build engine** (BuildKit or Buildah), on the building machine — with rootless Podman, in your user storage. It is mounted into the build container **during the instruction only**, then unmounted: nothing is written to a layer, so nothing in the image. A `VOLUME` is the opposite: a declaration in the image which, at **run time**, creates a volume for the container; it has no effect during the build. Without `# syntax=docker/dockerfile:1`: under Docker (recent versions, BuildKit by default), `--mount` works anyway with the current stable syntax; the line only served to force a newer frontend version. Under Podman, the line is **ignored** (Buildah has no frontend) and `--mount` works natively.

Why. The cache mount is a build-engine mechanism; the `VOLUME` a runtime mechanism. They only share the word "mount".

Nuance. The cache mount is not shared between machines or users, and it is not invalidated by content: a corrupted Maven repository stays there. `podman system prune --build-cache`... does not exist yet: you delete the storage or use `id=` to switch cache.

Example.

```
podman build --no-cache -f Dockerfile.cache -t c . 2>&1 | grep dep-    # markers accumulate
from one build to the next
podman run --rm c ls /root/.m2                                         # absent from the image
```

Question 11 — "Multi-stage is useless for Angular"

Answer. Without multi-stage, the final image is the image in which `ng build` ran: `node:22-alpine` (~170 MB) **plus** `node_modules` (500 MB to 1 GB) **plus** the TypeScript sources **plus** `dist/` — and you still need to add a server to serve `dist/`. With multi-stage: `nginx:alpine` (64 MB) plus a few MB of static files. The gap is 10 to 20 times, and the content changes in nature: no more Node, no sources, no build dependencies.

Why. The fact that the *result* is static is precisely the argument **for** multi-stage: since execution needs nothing of what served to build, you might as well keep nothing.

Nuance. Without a container, a team can also run `ng build` in CI and copy `dist/` into an nginx image in a single step (`COPY dist/ /usr/share/nginx/html`). That is a "multi-stage" whose first stage is CI — valid, but the build is no longer reproducible from the Dockerfile alone.

Example.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'   # 64.2 MB
against 167 MB
```

Question 12 — `/app/dist: no such file or directory`

Answer. The `build` stage has `WORKDIR /src`: the build produces `/src/dist`, not `/app/dist`. Fix: `COPY --from=build /src/dist/<project>/browser /usr/share/nginx/html` (the sub-folder depends on the Angular version and the project name). To diagnose without guessing: `podman build --target build -t dbg .` then `podman run --rm dbg find / -name index.html -path '*dist*'`.

Why. `COPY --from` copies from the stage's file system, with **absolute** paths of that stage. A `WORKDIR` or `dist/` structure error is invisible until you look inside.

Nuance. Since Angular 17, the default output is `dist/<project>/browser/`; before, `dist/<project>/`. `--target` avoids relying on memory.

Example.

```
podman build --target build -t dbg . && podman run --rm dbg ls -R /src/dist | head
```

Question 13 — `RUN mvn test` in the Dockerfile

Answer. In the build stage, **after** compilation and **before** `package` (or in a single `mvn package` without `-DskipTests`): if a test fails, `RUN` fails, the build stops, no image is produced. Drawback: the tests run in an isolated build container — no JUnit report usable by CI (it is in a discarded stage, unless copied out with `--target` or `--output`), no easily reachable test database (Testcontainers needs an engine), and the image build time includes that of the tests, even when you only wanted to rebuild.

Why. The Dockerfile is a good guarantor ("no image without green tests") but a poor reporting tool.

Nuance. The common compromise: CI runs the tests **and** the image build in two jobs, with the build conditioned on the tests succeeding; the Dockerfile keeps `-DskipTests` to stay fast. You get the report and the guarantee, at the price of a dependency on CI.

Example.

```
RUN mvn -q test            # red -> the build stops here
RUN mvn -q package -DskipTests
```

Question 14 — One 250 MB layer or five layers of 280 MB

Answer. The **280 MB image in five layers** deploys faster on a code update: only the volatile layer (a few MB) is transferred, the four others are already on the nodes and in the registry. The 250 MB single-layer image retransfers 250 MB at every version. The answer reverses when the nodes have **nothing** (first deployment, new node, emptied registry, or a tagging strategy that changes everything every time): there, 250 < 280, and the single layer wins — barely.

Why. The transfer cost is that of the missing layers, not of the image. Layer stability is worth more than their number.

Nuance. `--squash` (Buildah) or a minimal base can bring the 280 MB down to 250 without losing the layers: the two criteria are not exclusive. And the gain only exists if the stable layers are **bit-for-bit identical** from one build to the next — build reproducibility required (no unpinned `apt-get update` in a "stable" layer).

Example.

```
podman push registry.internal/myapp/api:1.5.1     # stable blobs: instantaneous; only the code layer is copied
```